

# Madalina-Eleonora Gheorghe

Senior UI / Frontend Engineer · Angular & TypeScript | UX-driven product thinking

Bucharest, Romania | [madalinaeleonora.gheorghe@gmail.com](mailto:madalinaeleonora.gheorghe@gmail.com) | +40 768 792 728

LinkedIn: [in/madalinaeleonorag](https://www.linkedin.com/in/madalinaeleonorag) | Portfolio: [madalinaeleonorag.github.io](https://madalinaeleonorag.github.io)

Extended version — per-project detail. A two-page version is available on request.

## PROFESSIONAL SUMMARY

UI Engineer with **7+ years** of software engineering, building complex enterprise web applications in Angular and TypeScript: micro-frontend estates, data-heavy administration interfaces, and the shared component and design-token layers underneath them. What I reach for first is how a product should *work and feel* for the person using it. I've owned UI/UX direction end to end, including full decision-making authority over a fraud platform's UI/UX at Cognizant, and colleagues have twice called me the team's "unofficial screen designer". I work at the seam between engineering, UX and product: challenging requirements so stories come out better structured, deciding what an interface needs to do, and making complex, permission-heavy workflows feel simple. I measure before proposing a refactor, and write the guard test that keeps the decision true afterwards.

## CORE EXPERTISE & SKILLS

- Product & UX:** UI/UX ownership, requirements analysis & discovery, wireframing, user flows & states, responsive design, accessibility (hands-on WCAG 2.1 / ARIA), Figma, design tokens & theming.
- Architecture & Frontend:** Angular (v2 to v21), TypeScript, JavaScript (ES6+), semantic HTML, CSS, Sass, Micro-Frontends (MFE), Module Federation, Nx monorepos, reactive state (Signals, NgRx SignalStore), zoneless Angular, RxJS, WebSockets, React, Vue.js (early projects), AG Grid, Kendo UI, Angular Material 3 (and the M2 migration to it), shared component library and design system maintenance, GraphQL, Ionic / Capacitor, AEM.
- Backend-adjacent, Testing & CI/CD:** REST APIs, OpenAPI / SDK generation, npm library packaging & publishing (ng-packagr, artifact registries), Keycloak / OIDC, Java (basic), Thymeleaf, AWS, Jest, Jasmine, unit testing, contract & snapshot testing, code review, Git, BitBucket, GitHub Actions, Docker, ArgoCD, Jenkins, Nexus, OpenShift.
- AI-assisted development:** Claude Code (CLI agents), MCP tooling, AI-assisted refactoring, code reading and code review.
- Leadership & ways of working:** E2E delivery ownership, Agile / Scrum, sprint planning, technical interviewing & hiring input, cross-team mentoring, stakeholder communication, written technical documentation.
- Languages:** Romanian (native), English (full professional proficiency).

## PROFESSIONAL EXPERIENCE

**SS&C Technologies — UI Engineer**

Hybrid, Bucharest · Sep 2025 – Present

*Financial-services administration platforms. The organisation is consolidating: for years every client got its own copy of the same application, and the direction now is one configurable platform serving all of them instead of separate codebases to keep in step. Five products carry that work and I build across all of them, in the Digital Technology and Development team, alongside a lead who holds the business overview and brings me the expectations. Around **90 people** in the Romanian organisation work on the same estate.*

**Recognised in a call across the whole services group** for having been through every one of the projects and improved them, and named one of the people it relies on.

## Access & Permissions Platform (RBAC)

Nov 2025 – Present

Everyone in the organisation uses this to enrol and administer the people and the client companies who have access to our applications, and to change a role when it is wrong. The application is itself governed by the thing it administers: what any one person can do inside it depends on their own role, so some reach all of the functionality and others only the part their role allows. Production holds the real client companies and every enrolled user, so seeing the true state of someone's access is the whole job. It is the furthest along of the five products, and like the rest of them still taking improvements and new feature requests.

**Stack:** Angular 21 · TypeScript · Signals · NgRx SignalStore · Micro-Frontends (MFE) · Module Federation · Nx · RxJS · OpenAPI · Keycloak / OIDC · Angular Material · Sass · Jest · GitHub Actions · Figma · WCAG 2.1 / ARIA

- **The project already existed and could not be maintained.** Organisational changes brought these applications to Romania, so the Romanian organisation inherited a codebase that was unreadable and inefficient. The lead and the architect had already decided, before I joined the team, that rewriting and carrying on would cost less than repairing it. I arrived into that rewrite, and everything below is the work of carrying it out.
- The rewrite is **eleven applications behind one shell** in place of a single monolith, with each area loading only for a user entitled to it (Angular 21, Nx, Module Federation). The lead gave me one example of what a remote application should look like and we went from there. **Six of the eleven I built from scratch; the other five I inherited from other people, then fixed and improved.** There is now no part of it I have not worked in.
- **A few months ago the lead said in front of the team that I am the code owner**, and he no longer touches the code there. I had refactored and improved most of the application by then, taken it to Angular 21 and reworked the interface.
- Built the permission-import feature the team calls the golden set, so that configuring a brand-new client is a clone rather than a rebuild: import an existing client's permission set as a file, then adjust it. **The harder problem was that nobody wanted to use it.** Importing is destructive, and enough had been broken by earlier imports that repairing the damage had become its own recurring job. The import now shows exactly what it will change before changing anything, which is what got it used.
- **Built the delegated-access programme end to end on the frontend:** one person acting for another under constrained entitlements: a permissions question as much as an implementation one. Multi-factor-authentication management is another of the sub-applications I built out, over Keycloak and OIDC.
- **The hardest part was never the development.** It was learning what the business words mean: what a ManCo is, what the product is, how capabilities link together. The code was the part I already knew how to do.

## Shared component kit, layout library & Material 3 theming

Nov 2025 – Present

The Material components behind the permissions platform, built alongside it from the start and improved since for ARIA and keyboard behaviour. Seven applications had been copy-pasting them from each other, so in **September 2026 I started pulling them out into one source of truth**, where a fix happens once instead of in every copy. It is deliberately an interim library of our own, until the company-wide component library now in progress is stable enough to install. On a design agreed with the lead.

**Stack:** Angular 21 · TypeScript · Angular Material 3 · design tokens · virtual scrolling · Sass · ng-packagr · Jest · Module Federation · WCAG 2.1 / ARIA

- **Moved the table filters into the table.** The problem was screen space: filters sat in a row of text boxes above the table and we needed those lines back for data. Put the filter into each column header instead. It touched every table in the application and all of it had to be retested, which is why **I converted one table first and showed it as a working example** rather than asking for the change in the abstract. People liked it, and we agreed together to convert the rest.
- **The consolidation argument only became winnable once I measured it.** Six findings, all checkable by anyone who doubted me. Among them: seven components whose “simplified” forks were *larger* than the original; three pairs sitting byte-identical in different applications and kept in step by hand; and forks that had gained useful features and never sent them home, so one application was quietly holding a better version than the one everyone treated as canonical. That last part is what made it more than a tidiness argument.

- The kit is **33 components** carrying **~1,050 tests** between them, built over the life of the permissions platform and now being reconciled into one place rather than rewritten, merging duplicate types instead of importing them twice. The blocker was a dialog service referenced in **60 files** that everyone assumed was too entangled to touch; I split its generic half from its domain-coupled half and costed the migration at **ten import lines**, and that number is what got it agreed. Virtual scrolling started with the lead; I carried it into the other lists and trees across the estate that were large enough to need it.
- **Designed the one component the estate did not have:** inline-editable table cells. Nothing saves optimistically, so the table and the page can never disagree about a value; the input goes read-only rather than disabled while saving, because a disabled input drops focus and a rejected save would reopen with the caret nowhere useful; and a verdict arriving after the user has pressed Escape is discarded rather than applied to a field they have walked away from.
- Migrated Angular Material **M2 to M3** behind a two-layer token system, **unprompted**: the lead and I improve the applications we work in whenever we can do it without breaking anything, so he saw this one after it was done. Every token falls back to a Material system property and then to a literal, because an application that has not installed a theme should still render correctly rather than render broken. **69 colour tokens** mapped deliberately, including the families Material has no equivalent for.
- A cleanup nobody enforces comes back, so the kit ships with the tests that keep it true: **356 assertions** that no stylesheet has reintroduced a colour literal or reached into another application, and **48** checked against every generated theme to catch a mistyped token name, which otherwise looks exactly like a deliberate fallback. Extracted five copy-pasted forks of the grid and layout SCSS into one framework-free published library and cut a remote's runtime entry bundle by **68.6%** (508 kB to 160 kB).

## Reference-data & content administration applications

Aug 2026 – Present

**Stack:** Angular 21 · TypeScript · Signals · NgRx SignalStore · Micro-Frontends (MFE) · Angular Material 3 · design tokens · Sass · Jest

- **Rebuilt the last thing standing in the legacy codebase.** One reference-data application was all that remained in the old permissions codebase. It was not in use any more, or not yet, but it could not simply be deleted, so I asked the architect what should happen to it: fold it into the permissions platform, or stand it up on its own. He decided it should be an application of its own. Parts of it had never been finished and there was nobody left to ask what they were meant to do, so I **went back to the original documentation and rebuilt the missing behaviour from that**. Finishing it is what lets the legacy codebase finally be archived: there is nothing left to migrate out of it.
- **Built the estate's single source of truth for applications and resources.** The idea was the architect's, raised in a daily call. I ran a spike to work out what it would need, built it in spare time, and own the code now.
- On the content-management application, which the lead owns, I contribute updates and fixes. Rebuilt its content-list, tag and category pages off two third-party rendering libraries and onto our own components, which **removed both dependencies** and gave those pages the estate's standard paging, filtering and accessibility behaviour without writing any of it again.

## Administrator configuration panel

Nov 2025 – Sep 2026

Where application administrators configure the platform, one micro-frontend per configuration area. Used by administrators only, not by the people the products serve.

**Stack:** Angular 20 · TypeScript · Signals · Micro-Frontends (MFE) · Module Federation · Sass

- **Data-transformation configuration: the first feature at the company, built with the lead** (Nov 2025). Set up a new micro-frontend from nothing and built the configuration of how a client's data is transformed on its way through the platform. The extensions it needed later went to another developer, by which point I was in full-time development on the permissions platform.
- **Built the frontend of the notifications configuration micro-frontend** (Jun–Sep 2026): creating a notification type, and the event that fires it.

- The notification widget itself sits in the applications rather than in the panel. **The lead started it; I refactored and redesigned it:** a different design for each notification status, its own redirect target so opening one goes straight to the thing it is about, and an animation as a new one arrives. Each notification carries a read and a delete action, except an important one, which renders differently and cannot be deleted.

## Platform TypeScript SDK

Jan 2026 – Present

The typed client every application in the estate talks to the backend through, generated from OpenAPI. The lead built it; I co-maintain it and run it day to day. Everyone builds against it, so when it stops, everyone stops.

**Stack:** TypeScript · OpenAPI · openapi-generator-cli · Angular · ng-packagr · npm publishing · GitHub Actions

- Keep it releasing on **every merge** rather than on someone remembering: **14 OpenAPI specifications** become **four published packages**, so no consumer waits on a manual publish. Added specifications and clients as new backend surface arrived, including delegated access and device registration.

## Design & accessibility

Across the estate

**Tools:** Figma · WCAG 2.1 / ARIA · VoiceOver · Angular Material 3 · design tokens

- **Product design.** Our designer produced a first draft for one application, covering where the header and sidenav should sit. I took it from there: designed the screens the draft did not cover, applied the language across every other application in the estate, and refined the original as the products grew.
- **Accessibility, as I build.** ARIA attributes, keyboard interaction, per-row labels on data tables so a screen reader announces which row an action belongs to, and focus management and restore in dialogs, written with the component rather than retrofitted. What a consuming application still has to do itself is documented per component, so the next person cannot miss it.

## Mentoring, interviewing & practice

Across the estate

**Tools:** Claude Code (CLI agents) · MCP-connected Jira & Confluence · Obsidian

- **Interviewing & hiring.** Ran the technical interview for two partner-vendor engineers joining the team in **August 2026**, and fed into the hiring decision.
- **Mentoring.** Mentoring engineers on our India-based team since new joiners arrived in **August 2026**, and the person peers come to when a change crosses code they do not know.
- **A verified engineering knowledge base.** Dense reference notes on the whole estate, every claim traceable to a file and line and re-checked rather than assumed. It is how I get productive in an unfamiliar application in hours, and the only reason the six measurements behind the consolidation argument existed at all.
- **AI-assisted delivery.** Automate the repetitive half of the work with AI tooling (Claude Code CLI, MCP-connected Jira and Confluence): scaffolding, mechanical cross-application refactors, reading unfamiliar code faster before a review, and drafting ticket and documentation updates.

## Cognizant — Frontend Developer & Interim Team Lead

Remote · Jan 2023 – Sep 2025

*Client: a Belgian retail bank. Two Angular applications built from nothing, both started with the tech lead. Frontend Developer, then Interim Team Lead after she left.*

### Banking fraud-detection platform

Jan 2023 – Sep 2025

Around 11 screens. Team at its largest: three frontend, four to five backend.

**Stack:** Angular 17+ · TypeScript · RxJS · WebSockets · AG Grid · Kendo UI · role-based access control (RBAC) · Sass · Jasmine · Jenkins · OpenShift · Nexus · BitBucket

- **Who it is for and what they do with it.** The bank's security agents, with two jobs. Watching whether the volume of fraud is staying inside a safe limit, so an emergency can be raised if it is not. And reviewing individual suspicious transactions: reading the customer's history and deciding to let it through, block it, or telephone the customer. A separate screen covers the transactions the system blocks automatically.

- **Started the application with the tech lead**, the only lead across both frontend and backend and therefore in meetings most of the time. Learned the architecture and the business rules from her and took over the code progressively, and took her position when she left on maternity leave. Being the only person who knew both the codebase and the business rules, I became the one who spoke in the business meetings.
- Built **role-based access control (RBAC)** across the application, and decided *with the business* which roles should be able to reach which screens. That is a permissions conversation as much as an implementation.
- Integrated AG Grid so senior agents could reclassify transactions in bulk across **50,000+** records, for instance from fraud to not-fraud. A bulk action was never applied blindly: it had already passed the automated system, then business rules in the frontend, then the same rules again in the backend. Anything that should not be reclassified was held back and routed to be handled another way, including recording whether the customer had been telephoned. **Agents could not get it wrong**, which was the entire point.
- **I made the interface decisions, but had to earn them.** The business analysts and product owners had mapped the flows from a BA's point of view. I argued the case from the point of view of an agent opening the screen for the first time with no background in the business, and from what was technically sensible, over many calls of pros and cons. It ended with them bringing me into the sessions with the **actual agents and their department managers**, so I could take feedback on requests directly. A process manager called me the team's "unofficial screen designer"; a manager review listed UX design among the tasks I picked up outside the role.
- Wrote guidance and descriptions throughout the interface, because the agents using it were often new hires or transfers from other departments who had to be productive without training.
- Added **WebSockets** to the screens where waiting for a refresh was the wrong behaviour, so an agent saw a fraud status change or a moving statistic as it happened.
- Releases went out every two to three months, at **5am on a Saturday. Release owner from the third release onwards**, with no production incidents across the releases I owned.
- **Became Interim Team Lead without being asked** — it was assumed and encouraged, particularly from the client side. Raised the Angular version, ran refinements and rewrote feature descriptions so they were unambiguous for colleagues, and reviewed the team's code. Worked with two under-delivering colleagues until their throughput went from **one story in two weeks to one story in three days**.
- Finished as the only frontend developer on it alongside one backend developer, onboarding a new frontend joiner.

## Employee access-management system

2024 – Sep 2025

The second application the two of us started from nothing.

**Stack:** Angular · TypeScript · RxJS · Sass · Jasmine · Jenkins · OpenShift

- Started it with the same tech lead. It administers the bank's own staff access: finding a user, granting or replacing their role, and editing their details.
- Took it to **MVP maturity**, at which point it was handed to a separate team under a new lead. Kept helping them afterwards, and the new lead in particular, with the harder Angular problems and with business decisions.
- When that team was away for a month, **I was the person responsible for it**, as the only person who still knew all of it.

**Internal banking application — sole frontend developer**

Feb 2022 – Dec 2022

Client and loan management for the staff of a banking client, built on Mambu's core-banking platform and inside its constraints. Also the first time I worked in React.

**Stack:** React · TypeScript · Mambu · Java · Thymeleaf · AWS · REST APIs

- The **only frontend developer for eleven months**, on a project already started by someone else that I picked up and carried, working directly with every other role rather than through a frontend team. Code review came from backend-leaning colleagues and from a principal frontend lead who did not work in the codebase, so I set and held the frontend's standards.
- Built the flows bank staff use to manage client profiles, onboard clients, and create and refinance loans: identity and eligibility checks, then amount and term selection. All financial calculation sat in the backend, so this was flow, validation and state work rather than computation.
- **Took over the Java PDF reporting** the bank's staff depended on, a customer's transaction history. API changes were needed and the backend developers were committed elsewhere, so I made them myself. Then, seeing the generated file looked poor, I took on the document too: **pagination, a header, the footer the bank was legally required to carry, and proper alignment**. Nobody asked for that second part.
- Investigated and fixed backend defects when backend developers were unavailable, keeping delivery continuous through a resourcing gap.
- Mambu's out-of-the-box interface was unattractive, and I improved its appearance as far as the platform allowed. Learning React while working inside someone else's platform constraints was the genuine difficulty of that year.
- Became the point of contact for the frontend within weeks, and challenged input data and technical approaches so requirements came out better structured, feedback a team lead singled out in a review. In practice that mostly meant validation rules, and insisting an interface be usable at first glance rather than after training.

**Insurance purchase & cover-upgrade platform — Deloitte's own product**

Oct 2021 – Jan 2022

An MVP for a product Deloitte intended to own and sell rather than a client delivery. Two frontend developers, four to five backend, one designer.

**Stack:** Angular · TypeScript · RxJS · Sass

- Built the stepped flow for taking out an insurance policy, and the piece I found most interesting in it: **mid-term adjustment**. A customer who discovered that their live policy did not cover something they needed could upgrade it mid-term and pay only the difference. **Upgrades only, never downgrades**, a rule the interface had to be designed around rather than validated against.
- Built for **untrained end users rather than insurance professionals**. Whether the eventual audience was agents or customers buying directly was never fully settled, so we designed for the harder of the two and made it usable with no training.
- Spoke with the designer **every day** as the frontend's feasibility voice: what was achievable, what was not, and what a given idea would cost to build. The product came out more consistent and easier to use because of those conversations, on feedback the designer attributed to me. I did not own the design, but shaped it daily.
- The MVP ran to its conclusion in January 2022 and I moved on to the banking application below.

IBM iX. Two client engagements over three years: Munich Re, then Albertsons. Joined as an intern, left as a developer other people were onboarded onto.

**Albertsons: e-commerce platform and CMS across six grocery brands**

Apr 2020 – Oct 2021

One AEM-based CMS driving six brand websites for a major US grocery group, built by roughly 40 developers in Romania working alongside a similar number on the client side.

**Stack:** Angular · TypeScript · RxJS · AEM (Adobe Experience Manager) · vanilla JavaScript · Sass · CSS · feature toggles · Jasmine · Jenkins · ARIA / VoiceOver

- On the product-browsing and cart team: product lists and carousels, add-to-cart, the product detail page, and the mini-cart in the site header. Teams owned separate applications that met in the browser and shared state through local and session storage. It was an early, hand-rolled version of the micro-frontend integration I now do properly, and the constraints were the same ones.
- **Built roughly 90% of the coupon experience**, the feature shoppers touched most. On a search or category page, a product with eligible offers shows an “x coupons available” link; opening it slides in the offers with their terms (“20% off when you buy three”), and applying one strikes through the original price and shows the new one beside it. The same control repeats on the product page and inside the mini-cart, where applying a coupon turns green with a check, updates the order total **in real time**, and reports how many products the offer covers. Coupons could also be redeemed by code from the site or the customer’s profile. The same machinery carried **BOGO** (buy one, get one) offers, where applying the code added a second unit at no charge.
- Built the store-location behaviour: a shopper enters a postal code or address and picks their nearest store, and that one choice drives stock visibility, delivery pricing, which in-store promotions apply, and which items are genuinely available for pickup.
- Wired every feature I built behind the advertising team’s **feature toggles**, so they could switch a component between its old and new version from their own settings with no release. The cost was real: most of the components I built existed as **two live versions at once**, and both had to keep working.
- One CMS drove six brands, so a change landed everywhere simultaneously, and a bug could reproduce on one brand and not another. Working out the blast radius before changing anything was the daily discipline.
- Wrote some functionality in plain JavaScript rather than Angular, either because it was a small AEM-authored component or because it was simply the leanest option: US shoppers on poor connections had to get a working page regardless of signal. Performance was tracked in the network panel, on total payload and CSS and JavaScript weight.
- **Accessibility was client-mandated and independently verified**, not self-assessed. ARIA, keyboard interaction and screen-reader behaviour were checked by dedicated testers on every ticket, and I tested with VoiceOver while building. Writing accessibility in rather than retrofitting it started here.
- Helped diagnose a **production outage that stopped checkout** on one brand: the checkout team had introduced a CSS class with the same name as one of ours, style encapsulation did not hold, and a button ended up hidden, so shoppers could not pay for around an hour. I was on call, and found the cause working through it with the senior engineers. Scoped styles and token contracts are a correctness problem, not a tidiness one.
- Built personalised reorder: repeating a previous order in one click from order history, plus a homepage carousel of previously bought items so regular shoppers could find their usual products without searching.
- Owned daily builds, led feature demos, mentored **one to two junior developers** (with feedback on that from both the junior and the manager), and was the point of contact for the client’s US team, working US hours from Romania for the duration. Separately, taught a colleague enough **Vue.js** to start a new project in it, and contributed to a web-analytics project by reviewing colleagues’ code and building reusable components.

Munich Re: offline-capable field-service app, document platform & device compliance

Mar 2019 – Mar 2020

Three products for a reinsurance client, all of them unit-tested.

**Stack:** Angular · Ionic · Capacitor · TypeScript · RxJS · Sass · offline-first storage & sync · Jamf · Jasmine

- One of **7–8 developers** on a tablet application for repair technicians working in customers’ homes with unreliable or no connectivity: check whether the appliance is covered, pull up the policy, record what was repaired, photograph it as proof, and capture the customer’s signature on the device. Everything captured had to be **legally valid**, which set the bar for validation and audit across the whole product.
- The offline storage-and-sync layer was our UI architect’s design. I built features on top of it: photo capture and attachment against a repaired item, new insurance entries, and history browsing by item or by customer, to designs from a client-side designer who set a genuinely demanding bar.
- Built the frontend for a **company-wide document platform**, a wiki-style single place for every document the organisation held, research included, at a conservative **10,000+** files. Search reached inside file *contents* as well as metadata, executed server-side. Available to everyone in the company.
- Took over an ageing internal application for company-phone enrolment and compliance (Jamf-backed device checks including patch state and weak passwords, mandatory for anyone with an enrolled company device) and **worked through its security vulnerabilities one at a time by hand**, tracing each package dependency to its fix. Slow, unglamorous work, and the reason the application could stay in service.
- Travelled to the client in Munich **three times, 3–5 days each**: technical discussions on how features should be built, face-to-face sessions with the product owner, demos, and working alongside the client’s own developers.

Internship: Angular from zero to client tickets

Dec 2018 – Feb 2019

First professional role, on the reinsurance field-service product above.

**Stack:** Angular · Ionic · TypeScript · RxJS

- Joined with no professional JavaScript experience, only Vue.js projects built at student competitions. Learned Angular and Ionic through IBM’s training programme, and by month two was picking up real tickets on the client’s product and joining the daily calls.
- Offered an indefinite-term contract at the end of the three months, on the client’s feedback.

## EDUCATION, CERTIFICATIONS & AWARDS

**Romanian-American University** — Faculty of Managerial Informatics, Bucharest

Master of Science, Computer Science for Economics · Grade: **9.66/10**2018 – 2020

Bachelor of Science, Computer Science for Economics · Grade: **9.50/10**2015 – 2018

### Certifications

Web Accessibility Basics · O’ReillyFeb 2026

Figma UI UX Design Essentials · FigmaSep 2025

GitHub Copilot for Angular developers, knowledge-based assessment · CognizantMay 2024

Think Like a Hacker · IBMSep 2021

Watson and Cloud Foundations · IBMJan 2021

IBM Mentor · IBMDec 2020

Enterprise Design Thinking Practitioner · IBMMar 2020

**Awards:** 1st place, Web & Mobile Challenge, DevHacks · 2nd place, RAU DevHack hackathon

## SELECTED FEEDBACK

---

“Not only you help us with designing and developing screens for our end user, you also think with us to find the best suitable solution which is smart and user friendly, and even propose solutions we did not think of ourselves that meet the business requirements. This is more than we would expect from a front-end developer.”

Process Manager · Cognizant, Nov 2024

“We appreciate your ownership, your involvement, your E2E view, the extra tasks you picked up outside of your role: UX design ... how you take up a key position after replacing the old Tech Lead, you to think along with business about problems and solutions.”

Manager · Cognizant, Nov 2024

“During refinements and clarifications, she challenges the approach and the input data, and thus we managed to increase the quality of the features’ description, resulting in better structured stories and acceptance criteria.”

Team Lead · Deloitte Digital, Apr 2022

---

Project descriptions are deliberately generic: client, product and internal repository names are omitted. Full performance reviews from  
Technical Leads & Project Managers: [madalinaeleonorag.github.io](https://madalinaeleonorag.github.io)